

PROYECTO FINAL

Red Social Académica con Estructuras de Datos en C++ y Benchmark Comparativo con Java

Estudiante: RONY BARRIOS

PROGRAMACIÓN III

Fecha: 29/05/2026

Repositorio GitHub: <https://github.com/RonyBarriosGT55/Proyecto-final/tree/main>

Video YouTube:

<https://www.youtube.com/watch?v=hX5OyZqt5Qs&list=PLsMjcaKtrpRMzTLXVXV6pVdy9BfgVeoUb&index=1>

1. Introducción

Este informe presenta las evidencias del proyecto final Red Social Académica Universitaria, desarrollado en C++ mediante estructuras de datos manuales. El sistema permite cargar estudiantes desde un archivo CSV, registrar estudiantes, buscar por ID, administrar proyectos académicos, conectar estudiantes mediante un grafo, ejecutar recorridos BFS y DFS, consultar ranking académico mediante un Árbol AVL y generar un benchmark comparativo.

2. Objetivo general

Desarrollar una aplicación en C++ que simule una red social académica universitaria, aplicando estructuras de datos avanzadas para almacenar, organizar, buscar y relacionar información de estudiantes. Además, se incluye un módulo de benchmark comparativo con Java para analizar diferencias de rendimiento y abstracción entre estructuras manuales y estructuras nativas del lenguaje.

3. Estructuras implementadas

Tabla Hash

Se utiliza para indexar estudiantes por `student_id`. Permite insertar, buscar y eliminar estudiantes, reportar colisiones y mostrar el factor de carga. La resolución de colisiones se realiza mediante encadenamiento separado.

Árbol AVL

Se utiliza para el ranking académico basado en `skill_score`. Permite insertar estudiantes, mostrar ranking ordenado, buscar por score, mostrar altura del árbol y realizar recorrido InOrden.

Grafo no dirigido

Representa conexiones académicas entre estudiantes. Permite conectar estudiantes, mostrar conexiones, verificar conexiones directas, ejecutar BFS, ejecutar DFS y encontrar ruta más corta como mejora avanzada.

Lista enlazada

Cada estudiante posee una lista de proyectos o publicaciones académicas. Cada proyecto contiene `project_id`, título, descripción y año de finalización.

Pila o cola manual

El proyecto utiliza estructuras manuales de apoyo para recorridos y control de acciones, evitando sustituir las estructuras obligatorias con colecciones STL.

4. Evidencias de pruebas funcionales

Prueba 01. Menú principal del sistema

Objetivo	Verificar que el sistema presente el menú principal con todos los módulos solicitados.
Procedimiento	Ejecutar el programa C++ y observar el menú principal.
Resultado esperado / obtenido	El sistema muestra opciones para cargar CSV, registrar, buscar, eliminar, administrar proyectos, grafo, AVL, estadísticas, benchmark, salida y mejora avanzada.
Estructura evaluada	Interfaz de consola del sistema principal.

```
*****
*          RONY ALEXANDER BARRIOS  9941 99 127          *
*          RED SOCIAL ACADEMICA UNIVERSITARIA - C++      *
*          *****                                     *
* Proyecto Final: Estructuras de Datos Manuales y Benchmark Comparativo *
*          *****                                     *
*****

          MENU PRINCIPAL DEL SISTEMA

+-----+ +-----+
| MODULO ESTUDIANTES | | MODULO RED / AVL |
+-----+ +-----+
| [ 1] Cargar estudiantes CSV | | [ 7] Conectar estudiantes |
| [ 2] Registrar estudiante   | | [ 8] Verificar conexion   |
| [ 3] Buscar estudiante por ID | | [ 9] Mostrar conexiones  |
| [ 4] Eliminar estudiante    | | [10] Recorrer red con BFS   |
| [ 5] Agregar proyecto      | | [11] Recorrer red con DFS   |
| [ 6] Mostrar proyectos     | | [12] Opciones del AVL      |
+-----+ +-----+

+-----+ +-----+
| ESTADISTICAS | | HERRAMIENTAS |
+-----+ +-----+
| [13] Estadisticas estructuras | | [14] Generar benchmark C++ |
|                               | | [16] Ruta mas corta [MEJORA] |
+-----+ +-----+

+-----+
| [15] Salir del sistema |
+-----+

-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:
```

Figura 01. Evidencia de Menú principal del sistema.

Prueba 02. Carga masiva de estudiantes desde CSV

Objetivo	Comprobar que el sistema cargue correctamente los estudiantes base desde estudiantes.csv.
Procedimiento	Seleccionar la opción 1 y presionar ENTER para usar la ruta predeterminada del CSV.
Resultado esperado / obtenido	El sistema carga 10,000 estudiantes y reporta rechazos y tiempo total de carga.
Estructura evaluada	Carga de datos, Tabla Hash, Árbol AVL y Grafo inicial.

```
| ESTADISTICAS | | HERRAMIENTAS |
+-----+ +-----+
| [13] Estadísticas estructuras | | [14] Generar benchmark C++ |
| | | [16] Ruta mas corta [MEJORA] |
+-----+ +-----+

+-----+
| [15] Salir del sistema |
+-----+

-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:1
Ruta del CSV [ENTER para usar ./dataset/estudiantes.csv]:

Carga finalizada. Estudiantes cargados: 10000 | Rechazados: 0 | Tiempo: 11.4200 ms

Presione una tecla para continuar . . .|
```

Figura 02. Evidencia de Carga masiva de estudiantes desde CSV.

Prueba 03. Estadísticas iniciales de estructuras

Objetivo	Validar que el sistema muestre estadísticas internas después de cargar el CSV.
Procedimiento	Seleccionar la opción 13 después de cargar el dataset.

Resultado esperado / obtenido	Se muestran estudiantes cargados, conexiones, proyectos, colisiones hash, factor de carga, altura del AVL, tiempo de carga y acciones en historial.
Estructura evaluada	Hash Table, AVL, Grafo y Pila/Historial.

```

Nota: la opcion 10 es una mejora avanzada adicional
Seleccione una opcion del menu [1-16]:13
===== ESTADISTICAS =====
Total de estudiantes cargados: 10000
Total de conexiones creadas: 0
Total de proyectos registrados: 0
Total de colisiones hash: 0
Factor de carga hash: 0.4997
Altura actual del AVL: 14
Tiempo total de carga del dataset: 8.9714 ms
Acciones en historial/pila: 1
=====
Presione una tecla para continuar

```

Figura 03. Evidencia de Estadísticas iniciales de estructuras.

Prueba 04. Búsqueda de estudiante inexistente

Objetivo	Comprobar que el sistema valide correctamente la búsqueda de un ID que no existe.
----------	---

Procedimiento	Seleccionar la opción 3 e ingresar el ID 50001 antes de registrarlo manualmente.
Resultado esperado / obtenido	El sistema indica que el estudiante no fue encontrado.
Estructura evaluada	Tabla Hash.

```

+-----+ +-----+
| [13] Estadísticas estructuras | | [14] Generar benchmark C++ |
|                               | | [16] Ruta mas corta [MEJORA] |
+-----+ +-----+

+-----+
| [15] Salir del sistema |
+-----+

-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:3
3
ID a buscar:50001
50001
Estudiante no encontrado.

Presione una tecla para continuar . . .

```

Figura 04. Evidencia de Búsqueda de estudiante inexistente.

Prueba 05. Registro manual de estudiante

Objetivo	Registrar manualmente un estudiante nuevo dentro del sistema.
----------	---

Procedimiento	Seleccionar la opción 2 e ingresar ID 50001, nombre, carrera, semestre, GPA y skill_score.
Resultado esperado / obtenido	El sistema registra correctamente el estudiante y lo agrega a las estructuras necesarias.
Estructura evaluada	Tabla Hash, Árbol AVL, Grafo y Pila/Historial.

```

+-----+ +-----+
| ESTADISTICAS | | HERRAMIENTAS |
+-----+ +-----+
| [13] Estadísticas estructuras | | [14] Generar benchmark C++ |
| | | [16] Ruta mas corta [MEJORA] |
+-----+ +-----+

+-----+
| [15] Salir del sistema |
+-----+

-----
Nota: la opción 16 es una mejora avanzada adicional.
Seleccione una opción del menú [1-16]:2
ID del estudiante:50001
50001
Nombre completo:Pedro Garcia
Pedro Garcia
Carrera:Ingeniería en sistemas
Ingeniería en sistemas
Semestre:6
6
GPA/promedio:88.50
88.50
Skill score:180
180
Estudiante registrado correctamente.

```

Figura 05. Evidencia de Registro manual de estudiante.

Prueba 06. Búsqueda del estudiante registrado

Objetivo	Confirmar que el estudiante registrado pueda ser localizado por ID.
Procedimiento	Seleccionar la opción 3 e ingresar el ID 50001.
Resultado esperado / obtenido	El sistema muestra los datos del estudiante registrado.
Estructura evaluada	Tabla Hash.

```
| [13] Estadísticas estructuras | | [14] Generar benchmark C++ |
|                               | | [16] Ruta mas corta [MEJORA] |
+-----+ +-----+

+-----+
| [15] Salir del sistema |
+-----+

-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:3
3
ID a buscar:50001
50001
ID: 50001
Nombre: Pedro Garcia
Carrera: Ingenieria en sistemas
Semestre: 6
GPA: 88.5000
Skill score: 180
Proyectos registrados: 0

Presione una tecla para continuar . . .|
```

Figura 06. Evidencia de Búsqueda del estudiante registrado.

Prueba 07. Validación de estudiante duplicado

Objetivo	Demostrar que el sistema no permite registrar student_id duplicados.
Procedimiento	Intentar registrar nuevamente el ID 50001.
Resultado esperado / obtenido	El sistema muestra un mensaje de error indicando que ya existe un estudiante con ese ID.
Estructura evaluada	Validación de Tabla Hash.

```
+-----+ +-----+
| ESTADISTICAS | | HERRAMIENTAS |
+-----+ +-----+
| [13] Estadísticas estructuras | | [14] Generar benchmark C++ |
| | | [16] Ruta mas corta [MEJORA] |
+-----+ +-----+

+-----+
| [15] Salir del sistema |
+-----+

-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:2
*****
*          RONY ALEXANDER BARRIOS  9941 99 127          *
*          RED SOCIAL ACADEMICA UNIVERSITARIA - C++      *
*                                                         *
*****
* Proyecto Final: Estructuras de Datos Manuales y Benchmark Comparativo *
*****

ID del estudiante:50001
50001
Error: ya existe un estudiante con ese ID.

Presione una tecla para continuar . . .
```

Figura 07. Evidencia de Validación de estudiante duplicado.

Prueba 08. Agregar proyecto a estudiante

Objetivo	Agregar un proyecto académico al estudiante registrado.
Procedimiento	Seleccionar opción 5 e ingresar ID del estudiante, project_id, título, descripción y año.
Resultado esperado / obtenido	El sistema agrega el proyecto correctamente.
Estructura evaluada	Lista enlazada de proyectos.

```
| [ 2] Registrar estudiante | | [ 8] Verificar conexion | |
| [ 3] Buscar estudiante por ID | | [ 9] Mostrar conexiones | |
| [ 4] Eliminar estudiante | | [10] Recorrer red con BFS | |
| [ 5] Agregar proyecto | | [11] Recorrer red con DFS | |
| [ 6] Mostrar proyectos | | [12] Opciones del AVL | |
+-----+ +-----+

+-----+ +-----+
| ESTADISTICAS | | HERRAMIENTAS | |
+-----+ +-----+
| [13] Estadísticas estructuras | | [14] Generar benchmark C++ | |
| | | [16] Ruta mas corta [MEJORA] | |
+-----+ +-----+

+-----+
| [15] Salir del sistema | |
+-----+

-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:5
5
ID del estudiante:50001
50001
ID del proyecto:90001
90001
Titulo:Sistema Red Academico
Sistema Red Academico
Descripcion:Proyecto desarrollado en c usando estructuras manuales
Proyecto desarrollado en c usando estructuras manuales
Año de finalizacion:2026
2026
Proyecto agregado correctamente.
```

Figura 08. Evidencia de Agregar proyecto a estudiante.

Prueba 09. Validación de proyecto duplicado

Objetivo	Comprobar que no se permita repetir el mismo project_id en un estudiante.
Procedimiento	Intentar agregar nuevamente el proyecto 90001 al estudiante 50001.
Resultado esperado / obtenido	El sistema muestra un error indicando que ese project_id ya existe.
Estructura evaluada	Lista enlazada y validación de duplicados.

```
+-----+ +-----+
| ESTADISTICAS | | HERRAMIENTAS |
+-----+ +-----+
| [13] Estadísticas estructuras | | [14] Generar benchmark C++ |
| | | [16] Ruta mas corta [MEJORA] |
+-----+ +-----+

+-----+
| [15] Salir del sistema |
+-----+

-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:
5
5
ID del estudiante:50001
50001
ID del proyecto:90001
90001
Error: ese project_id ya existe para este estudiante.

Presione una tecla para continuar |
```

Figura 09. Evidencia de Validación de proyecto duplicado.

Prueba 10. Mostrar proyectos del estudiante

Objetivo	Visualizar los proyectos registrados para un estudiante.
Procedimiento	Seleccionar opción 6, ingresar ID 50001 y luego elegir mostrar todos los proyectos.
Resultado esperado / obtenido	El sistema lista el proyecto registrado con su título, descripción y año.
Estructura evaluada	Lista enlazada.

```
| [15] Salir del sistema |
+-----+

-----

Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:6
ID del estudiante:50001
50001

===== MODULO DE PROYECTOS =====
1. Mostrar todos los proyectos
2. Buscar proyecto por ID
3. Eliminar proyecto por ID
4. Regresar al menu principal
Seleccione opcion:1
1
Proyecto ID: 90001
Titulo: Sistema Red Academico
Descripcion: Proyecto desarrollado en c usando estructuras manuales
Anio: 2026
-----

Presione una tecla para continuar
```

Figura 10. Evidencia de Mostrar proyectos del estudiante.

Prueba 11. Buscar proyecto por ID

Objetivo	Buscar un proyecto específico dentro de la lista de proyectos del estudiante.
Procedimiento	Seleccionar opción 6, ingresar ID 50001, elegir buscar proyecto e ingresar 90001.
Resultado esperado / obtenido	El sistema muestra los datos del proyecto solicitado.
Estructura evaluada	Lista enlazada.

```
| [13] Estadísticas estructuras | | [14] Generar benchmark C++ | |
|                               | | [16] Ruta mas corta [MEJORA] | |
+-----+ +-----+

+-----+
| [15] Salir del sistema |
+-----+

-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:6
6
ID del estudiante:50001
50001

===== MODULO DE PROYECTOS =====
1. Mostrar todos los proyectos
2. Buscar proyecto por ID
3. Eliminar proyecto por ID
4. Regresar al menu principal
Seleccione opcion:2
2
ID del proyecto:90001
90001
Proyecto ID: 90001
Titulo: Sistema Red Academico
Descripcion: Proyecto desarrollado en c usando estructuras manuales
Anio: 2026

Presione una tecla para continuar . . .
```

Figura 11. Evidencia de Buscar proyecto por ID.

Prueba 12. Conexión entre estudiantes

Objetivo	Crear una conexión académica entre dos estudiantes.
Procedimiento	Seleccionar opción 7 y conectar 1001 con 50001.
Resultado esperado / obtenido	El sistema confirma que la conexión fue creada correctamente.
Estructura evaluada	Grafo no dirigido.

```
-----  
Nota: la opcion 16 es una mejora avanzada adicional.  
Seleccione una opcion del menu [1-16]:7  
7  
ID del primer estudiante:1002  
1002  
ID del segundo estudiante:50001  
50001  
Conexion creada correctamente.  
Estudiante 1002 conectado con estudiante 50001.  
  
Presione una tecla para continuar . . .|
```

Figura 12. Evidencia de Conexión entre estudiantes.

Prueba 13. Validación de conexión duplicada

Objetivo	Demostrar que no se permiten conexiones duplicadas.
Procedimiento	Intentar conectar nuevamente 1001 con 50001.
Resultado esperado / obtenido	El sistema informa que no se pudo conectar porque puede ser una conexión duplicada.
Estructura evaluada	Grafo no dirigido y validación de duplicados.

```
-----  
Nota: la opcion 16 es una mejora avanzada adicional.  
Seleccione una opcion del menu [1-16]:7  
ID del primer estudiante:1001  
1001  
ID del segundo estudiante:50001  
50001  
No se pudo conectar. Puede ser una conexion duplicada.  
  
Presione una tecla para continuar . . .
```

Figura 13. Evidencia de Validación de conexión duplicada.

Prueba 14. Validación de conexión consigo mismo

Objetivo	Comprobar que un estudiante no pueda conectarse consigo mismo.
Procedimiento	Seleccionar opción 7 e ingresar 50001 como origen y destino.
Resultado esperado / obtenido	El sistema muestra que no se permite conectar un estudiante consigo mismo.
Estructura evaluada	Validación del Grafo.

```
| [15] Salir del sistema |
+-----+

-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:7

*****
*          RONY ALEXANDER BARRIOS  9941 99 127          *
*          RED SOCIAL ACADEMICA UNIVERSITARIA - C++      *
*                                                         *
*****
* Proyecto Final: Estructuras de Datos Manuales y Benchmark Comparativo *
*****

ID del primer estudiante:50001
50001
ID del segundo estudiante:50001
50001
No se permite conectar un estudiante consigo mismo.

Presione una tecla para continuar
```

Figura 14. Evidencia de Validación de conexión consigo mismo.

Prueba 15. Mostrar conexiones de estudiante

Objetivo	Mostrar las conexiones directas del estudiante dentro del grafo.
Procedimiento	Seleccionar opción 9 e ingresar el ID 50001.
Resultado esperado / obtenido	El sistema lista los estudiantes conectados a 50001.
Estructura evaluada	Grafo no dirigido.

```

+-----+
| [15] Salir del sistema |
+-----+

-----

Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:9
9
ID del estudiante:50001
50001
Conexiones de 50001: 1002 -> 1001

Presione una tecla para continuar . . .
```

Figura 15. Evidencia de Mostrar conexiones de estudiante.

Prueba 16. Recorrido BFS

Objetivo	Ejecutar una búsqueda en anchura desde un estudiante seleccionado.
Procedimiento	Seleccionar opción 10 e ingresar 50001 como nodo inicial.
Resultado esperado / obtenido	El sistema imprime el orden de visita BFS.
Estructura evaluada	Grafo y Cola manual.

```

*          RED SOCIAL ACADEMICA UNIVERSITARIA - C++          *
*
*****
* Proyecto Final: Estructuras de Datos Manuales y Benchmark Comparativo *
*****

ID inicial para BFS:50001
50001
BFS desde 50001: 50001 1002 1001

Presione una tecla para continuar . . .|
```

Figura 16. Evidencia de Recorrido BFS.

Prueba 17. Recorrido DFS

Objetivo	Ejecutar una búsqueda en profundidad desde un estudiante seleccionado.
Procedimiento	Seleccionar opción 11 e ingresar 50001 como nodo inicial.
Resultado esperado / obtenido	El sistema imprime el orden de visita DFS.
Estructura evaluada	Grafo y Pila manual.

```
-----  
Nota: la opcion 16 es una mejora avanzada adicional.  
Seleccione una opcion del menu [1-16]:11  
ID inicial para DFS:50001  
50001  
DFS desde 50001: 50001 1001 1002  
  
Presione una tecla para continuar . . .|
```

Figura 17. Evidencia de Recorrido DFS.

Prueba 18. Submenú del módulo AVL

Objetivo	Mostrar las opciones disponibles para el Árbol AVL.
Procedimiento	Seleccionar opción 12 en el menú principal.
Resultado esperado / obtenido	El sistema muestra opciones para ranking, búsqueda por score, recorrido InOrden y regreso al menú principal.
Estructura evaluada	Árbol AVL.

```
| [13] Estadísticas estructuras | | [14] Generar benchmark C++ |
|                               | | [16] Ruta mas corta [MEJORA] |
+-----+ +-----+

+-----+
| [15] Salir del sistema |
+-----+

-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:12

=====
MODULO AVL
=====
1. Mostrar ranking academico por skill_score
2. Buscar estudiantes por skill_score
3. Mostrar recorrido InOrden del AVL
4. Regresar al menu principal
=====
Seleccione opcion:
```

Figura 18. Evidencia de Submenú del módulo AVL.

Prueba 19. Ranking académico AVL

Objetivo	Mostrar estudiantes ordenados por skill_score.
Procedimiento	En el módulo AVL, seleccionar opción 1.
Resultado esperado / obtenido	El sistema muestra el ranking académico basado en skill_score.
Estructura evaluada	Árbol AVL.

```
3425. ID: 5345 | Luis Hernandez 5345 | Score: 60848 | GPA: 78.4800
3426. ID: 2642 | Ana Lopez 2642 | Score: 60837 | GPA: 98.9700
3427. ID: 10750 | Valeria Barrios 10750 | Score: 60833 | GPA: 77.3300
3428. ID: 8047 | Pedro Castillo 8047 | Score: 60822 | GPA: 97.8200
3429. ID: 5344 | Maria Garcia 5344 | Score: 60811 | GPA: 78.3100
3430. ID: 2641 | Juan Perez 2641 | Score: 60800 | GPA: 98.8000
3431. ID: 10749 | Jorge Contreras 10749 | Score: 60796 | GPA: 77.1600
3432. ID: 8046 | Sofia Morales 8046 | Score: 60785 | GPA: 97.6500
3433. ID: 5343 | Carlos Ruiz 5343 | Score: 60774 | GPA: 78.1400
3434. ID: 2640 | Helen Conoz 2640 | Score: 60763 | GPA: 98.6300
3435. ID: 10748 | Elena Pineda 10748 | Score: 60759 | GPA: 76.9900
3436. ID: 8045 | Luis Hernandez 8045 | Score: 60748 | GPA: 97.4800
3437. ID: 5342 | Ana Lopez 5342 | Score: 60737 | GPA: 77.9700
3438. ID: 2639 | Byron Castellanos 2639 | Score: 60726 | GPA: 98.4600
3439. ID: 10747 | Pedro Castillo 10747 | Score: 60722 | GPA: 76.8200
3440. ID: 8044 | Maria Garcia 8044 | Score: 60711 | GPA: 97.3100
3441. ID: 5341 | Juan Perez 5341 | Score: 60700 | GPA: 77.8000
3442. ID: 2638 | Nancy Chamal 2638 | Score: 60689 | GPA: 98.2900
3443. ID: 10746 | Sofia Morales 10746 | Score: 60685 | GPA: 76.6500
3444. ID: 8043 | Carlos Ruiz 8043 | Score: 60674 | GPA: 97.1400
3445. ID: 5340 | Helen Conoz 5340 | Score: 60663 | GPA: 77.6300
```

Figura 19. Evidencia de Ranking académico AVL.

Prueba 20. Búsqueda por skill_score

Objetivo	Buscar estudiantes según su puntaje académico.
Procedimiento	En el módulo AVL, seleccionar opción 2 e ingresar el score deseado.
Resultado esperado / obtenido	El sistema muestra los estudiantes que coinciden con el skill_score consultado.
Estructura evaluada	Árbol AVL.

```
Presione una tecla para continuar en el modulo AVL...
```

```
=====
```

```
MODULO AVL
```

```
=====
```

1. Mostrar ranking academico por skill_score
2. Buscar estudiantes por skill_score
3. Mostrar recorrido InOrden del AVL
4. Regresar al menu principal

```
=====
```

```
Seleccione opcion:2
```

```
Ingresa el skill_score a buscar:180
```

```
--- BUSQUEDA POR SKILL_SCORE ---
```

```
ID: 50001 | Pedro Garcia | Score: 180
```

```
Presione una tecla para continuar en el modulo AVL...
```

Figura 20. Evidencia de Búsqueda por skill_score.

Prueba 21. Recorrido InOrden del AVL

Objetivo	Mostrar el recorrido InOrden del árbol balanceado.
Procedimiento	En el módulo AVL, seleccionar opción 3.
Resultado esperado / obtenido	El sistema muestra los estudiantes según el recorrido InOrden.
Estructura evaluada	Árbol AVL.

```
ID: 7694 | Score: 47761 | Mario Lemus 7694
ID: 10397 | Score: 47772 | Dulce Arana 10397
ID: 2289 | Score: 47776 | Jorge Contreras 2289
ID: 4992 | Score: 47787 | Diego Navarro 4992
ID: 7695 | Score: 47798 | Paola Ortiz 7695
ID: 10398 | Score: 47809 | Nancy Chamal |® 10398
ID: 2290 | Score: 47813 | Valeria Barrios 2290
ID: 4993 | Score: 47824 | Rosa Fuentes 4993
ID: 7696 | Score: 47835 | Sergio Aragon 7696
ID: 10399 | Score: 47846 | Byron Castellanos 10399
ID: 2291 | Score: 47850 | Karla Gomez 2291
ID: 4994 | Score: 47861 | Mario Lemus 4994
ID: 7697 | Score: 47872 | Dulce Arana 7697
ID: 10400 | Score: 47883 | Helen Conoz 10400
ID: 2292 | Score: 47887 | Diego Navarro 2292
ID: 4995 | Score: 47898 | Paola Ortiz 4995
ID: 7698 | Score: 47909 | Nancy Chamal |® 7698
ID: 10401 | Score: 47920 | Juan Perez 10401
ID: 2293 | Score: 47924 | Rosa Fuentes 2293
```

Figura 21. Evidencia de Recorrido InOrden del AVL.

Prueba 22. Ruta más corta entre estudiantes

Objetivo	Demostrar la mejora avanzada de ruta más corta entre dos estudiantes.
Procedimiento	Seleccionar opción 16 e ingresar 1001 como origen y 1002 como destino.
Resultado esperado / obtenido	El sistema muestra la ruta más corta y los grados de separación.
Estructura evaluada	Grafo y BFS aplicado a ruta mínima.

```
-----  
Nota: la opcion 16 es una mejora avanzada adicional.  
Seleccione una opcion del menu [1-16]:16  
  
*****  
*                                                                 *  
*          RED SOCIAL ACADEMICA UNIVERSITARIA - C++          *  
*                                                                 *  
*****  
* Proyecto Final: Estructuras de Datos Manuales y Benchmark Comparativo *  
*****  
  
--- RUTA MAS CORTA (GRADOS DE SEPARACION) ---  
ID del estudiante origen:1001  
  
ID del estudiante destino:1002  
  
Ruta mas corta (2 grados de separacion): 1001 -> 50001 -> 1002  
  
Presione una tecla para volver al menu...|
```

Figura 22. Evidencia de Ruta más corta entre estudiantes.

Prueba 23. Generación de benchmark C++

Objetivo	Generar el archivo cpp_results.csv con tiempos de ejecución.
Procedimiento	Seleccionar opción 14 del menú principal.
Resultado esperado / obtenido	El sistema genera cpp_results.csv con tiempos para HashTable, AVL, BFS y DFS.
Estructura evaluada	Benchmark C++ con chrono.

```
-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:14

*****
*                                     *
*          RED SOCIAL ACADEMICA UNIVERSITARIA - C++          *
*                                     *
*****
* Proyecto Final: Estructuras de Datos Manuales y Benchmark Comparativo *
*****

Benchmark C++ generado en: cpp_results.csv
Control interno: Hash encontrados=10000, AVL visitados=10000, BFS visitados=10000, DFS visitados=10000

Presione una tecla para volver al menu...|
```

Figura 23. Evidencia de Generación de benchmark C++.

Prueba 24. Eliminación de estudiante

Objetivo	Eliminar un estudiante de todas las estructuras relacionadas.
Procedimiento	Seleccionar opción 4 e ingresar el ID 50001.
Resultado esperado / obtenido	El sistema confirma eliminación en Hash, AVL, Grafo y proyectos asociados.
Estructura evaluada	Hash, AVL, Grafo y Lista enlazada.

```
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:4

*****
*                                                                 *
*          RED SOCIAL ACADEMICA UNIVERSITARIA - C++          *
*                                                                 *
*****
* Proyecto Final: Estructuras de Datos Manuales y Benchmark Comparativo *
*****

ID a eliminar:50001

Estudiante eliminado correctamente en Hash, AVL, Grafo y proyectos asociados.

Presione una tecla para volver al menu...
```

Figura 24. Evidencia de Eliminación de estudiante.

Prueba 25. Confirmación de eliminación

Objetivo	Verificar que el estudiante eliminado ya no exista.
Procedimiento	Seleccionar opción 3 e ingresar 50001 después de eliminarlo.
Resultado esperado / obtenido	El sistema indica que el estudiante no fue encontrado.
Estructura evaluada	Tabla Hash.

```
+-----+ +-----+
| [15] Salir del sistema |
+-----+

-----
Nota: la opcion 16 es una mejora avanzada adicional.
Seleccione una opcion del menu [1-16]:3

*****
*                                     *
*          RED SOCIAL ACADEMICA UNIVERSITARIA - C++          *
*                                     *
*****
* Proyecto Final: Estructuras de Datos Manuales y Benchmark Comparativo *
*****

ID a buscar:50001

Estudiante no encontrado.

Presione una tecla para volver al menu...|
```

Figura 25. Evidencia de Confirmación de eliminación.

Prueba 26. Resultados o estadísticas finales

Objetivo	Mostrar el estado final del sistema después de las pruebas.
Procedimiento	Seleccionar opción 13 o mostrar resultados finales después de la eliminación.
Resultado esperado / obtenido	El sistema muestra los datos finales de las estructuras.
Estructura evaluada	Estadísticas generales del sistema.

```
*****
*                                                                 *
*          RED SOCIAL ACADEMICA UNIVERSITARIA - C++          *
*                                                                 *
*****
* Proyecto Final: Estructuras de Datos Manuales y Benchmark Comparativo *
*****

===== ESTADISTICAS =====
Total de estudiantes cargados: 10000
Total de conexiones creadas: 0
Total de proyectos registrados: 0
Total de colisiones hash: 1
Factor de carga hash: 0.4997
Altura actual del AVL: 14
Tiempo total de carga del dataset: 11.3474 ms
Acciones en historial/pila: 6
=====

Presione una tecla para volver al menu...|
```

Figura 26. Evidencia de Resultados o estadísticas finales.

5. Benchmark C++ y Java

El benchmark permite comparar tiempos de ejecución entre las estructuras manuales desarrolladas en C++ y estructuras nativas utilizadas en Java. En C++ se comparan HashTable, AVL y recorridos del Grafo; en Java se utilizan HashMap y TreeMap para comparar con la funcionalidad de búsqueda y estructura ordenada.

No se encontraron archivos de resultados CSV. Se recomienda colocar `cpp_results.csv` y `java_results.csv` en la misma carpeta del script o en una subcarpeta del proyecto.

6. Validaciones obligatorias comprobadas

- No se permiten `student_id` duplicados.
- No se permiten conexiones duplicadas entre estudiantes.
- No se permite conectar un estudiante consigo mismo.
- No se permite agregar proyectos duplicados por `project_id` dentro del mismo estudiante.
- La eliminación del estudiante se refleja en Hash, AVL, Grafo y proyectos asociados.
- Los recorridos BFS y DFS inician desde un estudiante seleccionado por ID.
- El sistema genera `cpp_results.csv` mediante benchmark C++.
- El benchmark Java debe generar `java_results.csv` usando HashMap y TreeMap.

BENCHMARK COMPARATIVO EN JAVA

Esta sección documenta la ejecución del benchmark comparativo desarrollado en Java. El objetivo de este módulo es comparar el uso de estructuras nativas del lenguaje frente a las estructuras manuales implementadas en C++ dentro del sistema principal.

Estructuras utilizadas en Java

Para la comparación con la Tabla Hash de C++ se utilizó HashMap. Esta estructura permite almacenar pares clave-valor y realizar búsquedas eficientes usando el student_id como clave principal.

Para la comparación con el Árbol AVL de C++ se utilizó TreeMap. Esta estructura mantiene sus elementos ordenados por clave y funciona como una estructura balanceada del framework de Java. En este benchmark se usa para comparar operaciones de inserción y recorrido ordenado.

Evidencia 1. Ejecución del benchmark Java

En la siguiente evidencia se observa la ejecución del programa JavaBenchmark. El programa procesa 10,000 registros, mide el tiempo de inserción y búsqueda en HashMap, además del tiempo de inserción y recorrido en TreeMap. La ejecución finaliza correctamente y genera el archivo java_results.csv.

```
=====
BENCHMARK JAVA - RED SOCIAL ACADEMICA
=====
Registros utilizados: 10000

HashMap insert: 3.7693 ms
HashMap search: 2.9307 ms
HashMap encontrados: 10000
TreeMap insert: 18.4606 ms
TreeMap traversal: 4.7788 ms
TreeMap visitados: 10000

Archivo generado correctamente: java_results.csv
Contenido esperado:
language,operation,structure,records,time_ms
Java,insert,HashMap,10000,...
Java,search,HashMap,10000,...
Java,insert,TreeMap,10000,...
Java,traversal,TreeMap,10000,...

Process finished with exit code 0
```

Figura 1. Ejecución del benchmark Java y generación del archivo java_results.csv.

Evidencia 2. Archivo java_results.csv generado

La siguiente evidencia muestra el archivo java_results.csv generado por el benchmark. El archivo conserva el formato CSV solicitado, usando las columnas language, operation, structure, records y time_ms. Si Excel muestra los datos en una sola columna, se debe a la configuración regional del separador, pero el archivo mantiene el formato correcto separado por comas.

```
language,operation,structure,records,time_ms
Java,insert,HashMap,10000,3.7693
Java,search,HashMap,10000,2.9307
Java,insert,TreeMap,10000,18.4606
Java,traversal,TreeMap,10000,4.7788
```

Figura 2. Archivo java_results.csv generado por el benchmark Java.

Tabla de resultados del benchmark Java

Lenguaje	Operación	Estructura	Registros	Tiempo ms
Java	insert	HashMap	10000	3.7693
Java	search	HashMap	10000	2.9307
Java	insert	TreeMap	10000	18.4606
Java	traversal	TreeMap	10000	4.7788

Texto listo para copiar al informe oficial

El benchmark Java fue desarrollado como un módulo separado del sistema principal en C++. Para comparar la Tabla Hash manual se utilizó HashMap, ya que permite trabajar con pares clave-valor y realizar búsquedas rápidas mediante `student_id`. Para comparar la funcionalidad del Árbol AVL se utilizó TreeMap, porque mantiene los datos ordenados y representa una estructura balanceada del framework de Java.

Los resultados obtenidos muestran que HashMap realizó la inserción de 10,000 registros en 3.7693 ms y la búsqueda en 2.9307 ms. Por su parte, TreeMap realizó la inserción en 18.4606 ms y el recorrido ordenado en 4.7788 ms. Estos resultados permiten comparar el rendimiento de las estructuras nativas de Java con las estructuras manuales implementadas en C++.

Como conclusión, Java facilita la implementación mediante estructuras ya disponibles en su framework, reduciendo la cantidad de código necesario. Sin embargo, la implementación manual en C++ permite comprender con mayor profundidad el funcionamiento interno de las estructuras, como el manejo de nodos, colisiones, recorridos, ordenamiento y control de memoria.

Tabla comparativa de benchmark C++ vs Java

Operaciones evaluadas sobre 10,000 registros. C++ usa estructuras manuales; Java usa HashMap y TreeMap.

Operación	Comparación	C++	Tiempo C++ ms	Java	Tiempo Java ms	Diferencia ms	Resultado
Inserción por ID	Tabla Hash manual vs HashMap	HashTabl e	1.0486	HashMap	3.1989	2.1503	C++ fue más rápido por 2.1503 ms
Búsqueda por ID	Tabla Hash manual vs HashMap	HashTabl e	0.1369	HashMap	2.1454	2.0085	C++ fue más rápido por 2.0085 ms
Inserción ordenada	Árbol AVL manual vs TreeMap	AVL	1.8988	TreeMap	8.3955	6.4967	C++ fue más rápido por 6.4967 ms
Recorrido ordenado	Árbol AVL manual vs TreeMap	AVL	0.0375	TreeMap	2.6994	2.6619	C++ fue más rápido por 2.6619 ms
Recorrido BFS	Grafo manual en C++	Graph	0.3445	No aplica	No aplica	No aplica	Sin equivalente Java en el benchmark actual
Recorrido DFS	Grafo manual en C++	Graph	0.3327	No aplica	No aplica	No aplica	Sin equivalente Java en el benchmark actual

Interpretación: en las operaciones comparables del benchmark, C++ obtuvo menor tiempo de ejecución que Java. Las operaciones BFS y DFS corresponden al grafo manual en C++ y no tienen equivalente directo dentro del benchmark Java actual.

Fuente: archivos `cpp_results.csv` y `java_results.csv` generados por los módulos de benchmark del proyecto.

7. Conclusiones técnicas

La Tabla Hash permitió realizar búsquedas rápidas por `student_id`, demostrando la utilidad del acceso por clave.

El Árbol AVL permitió mantener un ranking académico ordenado por `skill_score`, conservando una altura controlada.

El Grafo no dirigido permitió representar conexiones académicas y ejecutar recorridos BFS y DFS.

La Lista Enlazada permitió asociar múltiples proyectos a cada estudiante de forma dinámica.

La mejora de ruta más corta agrega valor al sistema, ya que permite analizar grados de separación entre estudiantes.

La comparación con Java permite evidenciar ventajas de abstracción y facilidad de uso en estructuras nativas como `HashMap` y `TreeMap`.